SIMANTIK DISKOMINFO KABUPATEN BANDUNG
Sistem Manajemen Layanan Teknologi Informasi • Dokumen Resmi

Dokumen:
SIMANTIK_DECISIONS_AND_CHANGELOG.md
Tanggal Cetak: 29 Oktober 2026

SIMANTIK — Log Keputusan Arsitektur & Riwayat Perubahan (Changelog)

Sistem Manajemen Layanan Teknologi Informasi Diskominfo Kabupaten Bandung

Dokumen ini mendokumentasikan keputusan arsitektur utama (*Architecture Decision Records*), aturan bisnis yang telah ditetapkan, dan riwayat kronologis perbaikan serta penyelarasan yang dilakukan oleh tim rekayasa perangkat lunak Antigravity.

---

1. Keputusan Arsitektur Utama (*Architecture Decision Records*)

ADR-001: Pemisahan Fisik Web Layanan Publik dan Web Admin

- **Konteks**: Web Layanan digunakan oleh pemohon OPD publik di seluruh Kabupaten Bandung, sedangkan Web Admin digunakan secara internal oleh staf dan Tenaga Ahli Diskominfo.
- **Keputusan**: Frontend dipisahkan menjadi dua aplikasi web mandiri:
  - `frontend/service-web` (Port 5173): Portal publik, **100% tanpa koneksi langsung ke Supabase**. Klien publik tidak memegang API key database apa pun; seluruh interaksi dilakukan via NestJS API.
  - `frontend/admin-web` (Port 5174): Konsol internal, menggunakan Supabase Auth untuk verifikasi sesi staf Diskominfo dan memanggil NestJS API menggunakan Bearer JWT.
- **Konsekuensi Positif**: Meminimalkan risiko ekspos kredensial database ke publik dan menyederhanakan isolasi tata letak UI.

ADR-002: Backend NestJS sebagai Pemegang Otoritas Tunggal (*Single Source of Truth*)

- **Konteks**: Validasi aturan bisnis SPBE, transisi status, izin peran, dan hak akses berkas tidak boleh dapat dimanipulasi oleh klien peramban.
- **Keputusan**: Seluruh mutasi basis data dan akses Storage Supabase dieksekusi secara eksklusif oleh NestJS backend menggunakan Supabase `service_role` key. Hak akses kueri langsung dari role `anon` dan `authenticated` di-revoke secara permanen via PostgreSQL RLS (*Default-Deny Policy*).
- **Konsekuensi Positif**: Frontend tidak dapat melewati aturan bisnis; jika tombol disembunyikan di UI, backend tetap menjadi lapisan validasi wajib utama.

ADR-003: Integritas Rekam Jejak Audit Tidak Dapat Diubah (*Append-Only Audit Trail*)

- **Konteks**: Kepatuhan hukum dan audit SPBE mensyaratkan bahwa seluruh riwayat tindakan administratif (verifikasi, persetujuan, penugasan, transisi status) tidak boleh dapat diubah atau dihapus oleh siapa pun, termasuk administrator sistem.
- **Keputusan**: Dibuat trigger PostgreSQL `trg_prevent_audit_log_update_delete` yang membatalkan dan melempar eksepsi error `P0001` pada setiap percobaan `UPDATE` atau `DELETE` pada tabel `audit_logs`.
- **Konsekuensi**: Mutasi hanya diizinkan via `INSERT`. Pada skenario pembersihan darurat berizin khusus, trigger harus dinonaktifkan sementara di dalam transaksi tertutup lalu segera diaktifkan kembali.

ADR-004: Pelacakan Publik Tanpa Akun Menggunakan Timing-Safe Compare & HMAC Token

- **Konteks**: Pemohon OPD perlu melacak perkembangan permohonan tanpa beban keharusan membuat akun login di awal.
- **Keputusan**: Pelacakan menggunakan Nomor Tiket (`SIM-YYYY-XXXXXX`) dan Kode Pelacakan Rahasia (`TRK-...`). Verifikasi menggunakan `crypto.timingSafeEqual` pada digest SHA-256 (32 byte) untuk mencegah serangan timing attack. Sesi pelacakan sah mendapatkan token sesi HMAC-SHA256 berumur 15 menit. Percobaan gagal dibatasi oleh `TrackingRateLimiter` (maksimal 5 kegagalan berturut-turut sebelum terkena HTTP 429).
- **Konsekuensi**: Pelacakan publik aman, cepat, dan terhindar dari pembobolan kode via brute force.

ADR-005: Pemisahan Identifier Internal `TECHNICIAN` vs Label Tampilan `Tenaga Ahli`

- **Konteks**: Terdapat masukan dari pembimbing/stakeholder Diskominfo bahwa istilah formal bagi personel fungsional teknis Aptika Diskominfo adalah **"Tenaga Ahli"**, bukan "Teknisi". Namun, pengubahan identifier enum di database, endpoint API, dan skema autentikasi berisiko memicu kerusakan regresi luas.
- **Keputusan**:
  - **Lapisan Internal Tetap**: Enum database `public.profile_role`, variabel backend `TECHNICIAN`, endpoint `/technician/*`, dan guard `Roles('TECHNICIAN')` dipertahankan 100%.
  - **Lapisan User-Facing Diselaraskan**: Seluruh teks tombol, badge, navigasi sidebar, placeholder, dan pesan API berbahasa Indonesia diubah konsisten menjadi **"Tenaga Ahli"**.
- **Konsekuensi**: Kebutuhan terminologi pengguna terpenuhi tanpa risiko regresi arsitektur database.

ADR-006: Pemisahan Event Penugasan dan Transisi Status Pengerjaan (IN_PROGRESS)

- **Konteks**: Penugasan Tenaga Ahli oleh Super Admin sering disalahartikan sebagai pemicu otomatis transisi tiket ke status `IN_PROGRESS`. Namun, dalam tata kelola SPBE, penugasan dapat terjadi saat berkas disetujui (`APPROVED`), sedangkan pengerjaan teknis nyata baru dimulai saat Tenaga Ahli siap membuka lembar kerja dan memulai telaah teknis.
- **Keputusan**:

    - `assignTechnician` pada `assignments.service.ts` **hanya mencatat penugasan** ke tabel `assignments` (`is_current = true`), menambahkan pesan penugasan internal, dan mencatat jejak audit. Status permohonan tetap berada pada status `APPROVED` .
    - Transisi ke `IN_PROGRESS` adalah **aksi alur kerja eksplisit ("Mulai Pengerjaan")** yang dieksekusi via `POST /api/v1/admin/applications/:id/transitions` atau `/api/v1/technician/applications/:id/transitions` oleh `SUPER_ADMIN` atau `TECHNICIAN` penanggung jawab.

- **Konsekuensi**: Akurasi audit terjaga, beban kerja operasional tercatat jelas, dan Tenaga Ahli memiliki kendali penuh atas inisiasi pengerjaan sistem.

ADR-007: Standarisasi Rute Endpoint Lifecycle Kanonis (`/api/v1/applications/:id/lifecycle`)

- **Konteks**: Sebelumnya, `lifecycle.controller.ts` mendeklarasikan path `@Controller('api/v1/applications/:id/lifecycle')` . Karena `main.ts` menetapkan global prefix `app.setGlobalPrefix('api/v1')` , NestJS mendaftarkan rute sebagai `/api/v1/api/v1/applications/:id/lifecycle` dan client hook `useLifecycle.ts` menyertakan prefix redundan serupa.
- **Keputusan**:
    - Mendeklarasikan path `@Controller('applications/:id/lifecycle')` secara kanonis pada backend NestJS.
    - Menstandarkan seluruh 31 pemanggilan di `useLifecycle.ts` menjadi `/applications/${applicationId}/lifecycle/...` .
    - Mempertahankan `apiBaseUrl` frontend pada `http://localhost:3000/api/v1` , sehingga URL akhir di jaringan menjadi `/api/v1/applications/:id/lifecycle/...` .
    - Menambahkan regression test metadata rute pada `lifecycle.spec.ts` untuk mengunci path kanonis.
- **Konsekuensi**: Arsitektur rute REST API menjadi bersih, konsisten dengan controller lain, dan terbebas dari redundansi prefix ganda tanpa mengubah API base URL frontend maupun behavior lifecycle apa pun.

ADR-008: Pola Kompensasi Parsial Berbasis Identifier Terisolasi & Guard Konkurensi pada Pengiriman Revisi

- **Konteks**: Alur `submitRevision` melibatkan multi-step lintas subsistem: (1) upload berkas fisik ke Supabase Storage privat, (2) insert metadata berkas ke `application_documents` , (3) update status permohonan ke `VERIFICATION` , (4) insert riwayat status ke `application_status_history` , dan (5) insert pesan diskusi publik ke `messages` . Jika kegagalan terjadi di langkah (4) atau (5), status tiket sebelumnya dapat tertinggal di `VERIFICATION` sementara dokumen dibersihkan, menimbulkan desinkronisasi data. Selain itu, filter `.eq('current_status', ...)` tanpa memeriksa jumlah baris hasil mutasi tidak membuktikan baris benar-benar terupdate jika status telah berubah bersamaan oleh proses lain.
- **Keputusan**:

    - **Verifikasi Hasil Mutasi Baris via PostgREST**: Menambahkan klausa `.select('id')` pada kueri update maju (`applications`) dan update rollback. Membedakan 3 kemungkinan hasil: (a) Database Error (`error != null`), (b) Query Sukses namun 0 Baris Cocok (`data.length === 0`), dan (c) Tepat Satu Baris Terbarui (`data.length === 1`).
    - **Penanganan Bentrok Konkurensi 0 Baris**: Jika update maju menghasilkan 0 baris (status permohonan telah berubah dari `REVISION_REQUIRED`), proses revisi langsung dibatalkan dengan melempar `ConflictException` (HTTP 409). Blok kompensasi hanya membersihkan Storage dan dokumen milik sesi tersebut, tanpa menulis status history, tanpa menulis pesan, dan **tanpa melakukan rollback status** agar mutasi sah pihak lain tidak tertimpa.
    - **Rollback Aman & Pelaporan Batas Kompensasi**: Jika rollback status di blok catch menghasilkan 0 baris cocok (karena status permohonan berubah lagi di tengah jalan), sistem tidak mengklaim rollback sukses, melainkan mencatat peringatan secara aman (`logger.warn`) dan mempertahankan status terkini pihak lain.
    - **Identifikasi Spesifik via UUID**: ID untuk `application_status_history` dan `messages` dialokasikan secara eksplisit (`crypto.randomUUID()`) sebelum insert. Saat terjadi kegagalan parsial, blok `catch` hanya menghapus baris spesifik tersebut (`.eq('id', id)`), sehingga riwayat dan pesan terdahulu aman dari penghapusan tidak sengaja.
    - **Penegasan Batas Kompensasi vs ACID**: Solusi ini adalah kompensasi berbasis aplikasi (*best-effort compensating actions*), bukan transaksi ACID basis data. Transaksi atomik penuh memerlukan PostgreSQL RPC/Stored Procedure yang memerlukan izin perubahan skema.

- **Konsekuensi Positif**: Status, riwayat, pesan, dan berkas tetap sinkron pada kegagalan parsial, proteksi konkurensi aktif, dan data historis terisolasi total tanpa perubahan skema.

ADR-009: Validasi Ketat Base64 (RFC 4648) & Storage Key Acak Server-Generated UUID

- **Konteks**: Unggahan berkas pengajuan awal dan revisi mentransmisikan data berkas dalam representasi Base64. Pemrosesan bawaan Node.js `Buffer.from(str, 'base64')` secara diam-diam mengabaikan karakter non-alfabet, kesalahan padding, trailing garbage, serta bit padding non-kanonis (misal `AB==`). Selain itu, penggunaan nama berkas asli (`original_filename`) di dalam `storage_path` Supabase Storage rentan terhadap path traversal, eksfiltrasi nama berkas sensitif, tabrakan nama berkas berulang, dan masalah karakter khusus/Unicode di bucket S3.
- **Keputusan**:
    - **Dukungan Format Kontrak Fleksibel**: Menerima payload Base64 mentah maupun format Data URL standar (`data:<mime>;base64,<payload>`). Prefix Data URL diparsing secara eksplisit dan divalidasi MIME-nya.
    - **Validasi Sintaks & Kanonisitas Ketat Sebelum Decode**:
        1. Input wajib berupa string tidak kosong (menolak `null` , `undefined` , whitespace).
        2. Panjang payload Base64 wajib merupakan kelipatan 4 (`length % 4 === 0`).
        3. Alfabet dan padding wajib memenuhi regex RFC 4648 `/^(?:[A-Za-z0-9+/]{4})*(?:[A-Za-z0-9+/]{2}==|[A-Za-z0-9+/]{3}=|[A-Za-z0-9+/]{4})$/` .
        4. Setelah `Buffer.from()` , hasil buffer wajib tidak kosong (`buffer.length > 0`).

5. **Pemeriksaan Kanonisitas RFC 4648**: Menguji `buffer.toString('base64') === payload` untuk menolak bit sisa ilegal (seperti `AB==`) dan trailing garbage yang diabaikan mesin decode Node.js.

- **Storage Object Key Acak Buatan Server**:
  - Format Storage key: `${prefix}/${crypto.randomUUID()}.${safeExt}` (contoh: `applications/SIM-2026-123456/550e8400-e29b-41d4-a716-446655440000.pdf`).
  - Nama berkas asli (`original_filename`) **100% dihilangkan dari object key path Storage**, mengeliminasi risiko traversal dan sanitasi S3.
  - Ekstensi (`safeExt`) diekstrak secara ketat dari allowlist (`pdf`, `doc`, `docx`, `xls`, `xlsx`, `png`, `jpg`, `jpeg`).
  - Nama berkas asli disanitasi (maksimal 255 karakter, bebas karakter kontrol dan null byte, mempertahankan karakter aksen/Unicode natural) dan disimpan secara aman di basis data (`application_documents.original_filename`) hanya untuk kebutuhan tampilan antarmuka (UI).
- **Konsistensi Siklus Hidup**: `storage_path` UUID acak digunakan seragam dalam insert basis data, download/signed URL, dan penghapusan cleanup/rollback. Objek Storage lama tidak diubah atau dihapus.

- **Konsekuensi Positif**: Keamanan unggahan meningkat drastis, kebal terhadap manipulasi Base64 dan serangan path traversal, tanpa memerlukan perubahan skema basis data dan tanpa memutus kontrak antarmuka frontend.

ADR-010: Penerbitan Ulang Kode Pelacakan Berbasis Transaksi Atomik PostgreSQL & Otorisasi Eksklusif Superadmin

- **Konteks**: Jika pemohon OPD kehilangan kode pelacakan rahasia, sistem harus menyediakan mekanisme penerbitan ulang kode pengganti tanpa membuat tiket baru. Namun, pembaruan hash dan pencatatan audit tidak boleh dilakukan secara dua tahap (non-atomik) di sisi aplikasi karena kegagalan audit akan meninggalkan database dalam kondisi terubah tanpa jejak forensik. Selain itu, kode tidak boleh dapat diakses oleh peran selain Superadmin dan tidak boleh disimpan dalam bentuk teks terang (*plaintext*).
- **Keputusan**:
  1. **Tingkat Keacakan 128-bit**: Kode baru dibuat menggunakan `crypto.randomBytes(16).toString('hex').toUpperCase()`, menghasilkan 128 bit entropi kriptografis murni dengan format `TRK-<32-HEX-CHARS>`.
  2. **Penyimpanan Eksklusif Hash SHA-256**: Hanya digest SHA-256 dari kode baru yang disimpan ke kolom `tracking_code_hash`. Kode plaintext tidak pernah disimpan di tabel basis data dan tidak pernah dicatat ke dalam `audit_logs`.
  3. **Transaksi Atomik via PostgreSQL RPC**: Dibuat fungsi database transaksional `public.reissue_tracking_code_atomic(p_application_id, p_new_hash, p_admin_id, p_reason)` dalam migrasi `20261019161000_reissue_tracking_code_atomic.sql`.

    - Fungsi memperbarui `tracking_code_hash`, mengupdate `updated_at`, dan mencatat baris `audit_logs` dengan aksi `TRACKING_CODE_REISSUED` dalam **satu transaksi database ACID tunggal**.
    - Jika salah satu operasi gagal atau tiket berstatus terminal (`COMPLETED`, `CLOSED`, `REJECTED`, `CANCELLED`), seluruh transaksi di-rollback secara otomatis di level engine PostgreSQL.
    - Dilarang menyediakan fallback non-atomik: jika pemanggilan RPC gagal, operasi langsung dibatalkan dengan eksepsi eksplisit.
4. **Keamanan Eksekusi Fungsi Database**:
    - Fungsi dideklarasikan dengan `SECURITY DEFINER` dan `SET search_path = ''` dengan kualifikasi skema eksplisit (`pg_catalog.*`, `public.*`) guna mencegah *search path hijacking*.
    - Hak akses `EXECUTE` dicabut dari `PUBLIC`, `anon`, dan `authenticated`, serta diberikan eksklusif hanya kepada `service_role`.
5. **Otorisasi Ketat Superadmin**:
    - Endpoint `POST /api/v1/admin/applications/:id/reissue-tracking-code` diproteksi guard `@Roles('SUPER_ADMIN')` di backend NestJS. Identitas role dari payload frontend tidak dipercaya.
    - Role lain (`TECHNICIAN` atau pemohon publik) ditolak dengan HTTP 403 Forbidden.
6. **Protokol Tampilan Satu Kali (One-Time Display)**:
    - Kode baru hanya dikembalikan satu kali dalam respons API dan ditampilkan melalui modal dialog `ReissueTrackingCodeModal.tsx` di Web Admin.
    - Setelah modal ditutup, kode plaintext tidak dapat dilihat kembali oleh siapa pun. Petugas wajib menyalin kode dan menyampaikannya kepada pemohon OPD melalui kanal verifikasi resmi.
    - Kode lama seketika tidak berlaku karena hash di database telah digantikan secara permanen.

- **Konsekuensi Positif**: Integritas data dan jejak audit terjamin secara atomik, risiko pembobolan kode pelacakan terminimalisasi, dan hak akses penerbitan ulang terkendali secara formal.

---

2. Riwayat Perubahan Kronologis Antigravity

Berikut adalah rangkuman seluruh tahapan perbaikan dan penyelarasan yang telah dikerjakan secara berurutan:

Revisi 1 — Penyelarasan Masukan Pembimbing Diskominfo

1. **Terminologi Tenaga Ahli**:

- Menyelaraskan teks navigasi dan antarmuka dari "Teknisi" menjadi "Tenaga Ahli" pada Admin Web (`Sidebar.tsx`, `navigation.ts`, header, modal dialog).
- Menyelaraskan pesan API fallback pada `assignments.service.ts` ("Tenaga Ahli TI", pesan otorisasi penugasan, pesan teknisi tidak aktif).

2. **Peningkatan Pelacakan Publik (`TrackingPage.tsx` & `tracking.service.ts`)**:
   - Menambahkan panduan langkah berikutnya (*next-step advisory*) yang jelas bagi pemohon OPD.
   - Menampilkan status kewajiban pemohon (*action required by applicant*): jelas membedakan kapan pemohon harus menunggu dan kapan pemohon harus bertindak (misal saat status `REVISION_REQUIRED`).
   - Menyusun proyeksi timeline tahapan publik yang intensional dan aman, bukan raw dump log audit internal.
3. **Penyajian Tenaga Ahli Penanggung Jawab pada Detail Tiket**:
   - Menampilkan nama Tenaga Ahli penanggung jawab aktif secara transparan pada kartu detail tiket di `AdminApplicationDetailPage.tsx`.
4. **Pemisahan Konteks OPD vs Kajian Teknis**:
   - Memastikan catatan internal staf (`INTERNAL`) dan dokumen kajian arsitektur tidak bocor ke kanal pemohon publik luar.

Revisi 2 — Perbaikan Bug Beban Kerja Tenaga Ahli & "Pekerjaan Saya"

1. **Akar Masalah (Root Cause)**:
   - Tiket yang telah berstatus terminal masih dihitung sebagai tiket aktif untuk Tenaga Ahli.
   - Penyebabnya: `assignments.service.ts` dan `technicians.service.ts` sebelumnya menghitung agregasi beban kerja hanya berdasarkan `is_current = true`, tanpa memeriksa apakah permohonan tersebut masih berstatus aktif operasional atau sudah berstatus terminal.
2. **Solusi Implementasi**:
   - Menambahkan filter status terminal pada perhitungan beban kerja:

     ```
     const terminalStatuses = ['COMPLETED', 'CLOSED', 'REJECTED', 'CANCELLED'];
     // Beban kerja aktif = is_current: true DAN !terminalStatuses.includes(status)
     ```

   - Menambahkan filter serupa pada `getTechnicianMyWork` dan `getTechnicianWorkSummary` sehingga tiket terminal otomatis keluar dari antrean pengerjaan aktif Tenaga Ahli.
3. **Hasil Verifikasi**: Beban kerja tiket CLOSED berubah menjadi 0 tiket aktif, dan halaman Pekerjaan Saya hanya menampilkan tiket yang benar-benar aktif (`APPROVED` / `IN_PROGRESS`).

Revisi 3 — Kontrol Penugasan dan Lifecycle pada Tiket Terminal

1. **Tombol Penugasan Tiket Terminal (`AdminApplicationDetailPage.tsx`)**:

    - Memastikan tombol `+ Tugaskan Sekarang` dan `Ganti Tenaga Ahli` tidak muncul jika tiket berstatus terminal (`COMPLETED`, `CLOSED`, `REJECTED`, `CANCELLED`).
    - Backend `assignments.service.ts` memperkuat validasi utama dengan melempar `BadRequestException` jika ada percobaan menugaskan tiket terminal.
2. **Kontrol Pemindahan Lifecycle Stage (`workflow.engine.ts` & UI)**:
    - Membatasi transisi tahapan siklus hidup SPBE di backend (`validateStageTransition`) hanya diperbolehkan jika status aplikasi adalah `APPROVED` atau `IN_PROGRESS`.
    - Pada antarmuka `AdminApplicationDetailPage.tsx` dan `LifecycleWorkspace.tsx`, tombol `(Ubah)` tahapan otomatis disembunyikan jika tiket berstatus terminal.

Revisi 4 — Penyelarasan Label Role User-Facing

1. `AdminTechniciansPage.tsx`: Mengubah teks label menjadi `Role: Tenaga Ahli`.
2. `DashboardPage.tsx`: Mengubah teks sambutan menjadi `Sebagai Tenaga Ahli`.
3. **Batasan Terjaga**: Identifier internal enum `TECHNICIAN`, RBAC, dan struktur API tidak diubah sedikit pun.

Revisi 5 — Pembersihan Label Internal "Phase N" dari UI

Menghapus seluruh sisa label roadmap/milestone internal yang sebelumnya tampil pada antarmuka:

- `AdminApplicationsListPage.tsx`: Menghapus badge `Phase 07`.
- `AdminAssignmentsOverviewPage.tsx`: Menghapus badge `Phase 08`.
- `DesignSystemShowcasePage.tsx`: Mengubah badge `Phase 02 Baseline` menjadi label natural `Standar Desain UI`.
- `ServicesPage.tsx`: Mengubah teks persiapan berkas menjadi panduan alami tanpa penyebutan fase internal.
- `ModulePlaceholderPage.tsx`: Mengubah badge menjadi `Dalam Pengembangan`.

Revisi 6 — Pembersihan Data UAT Terkontrol & Pencadangan Terisolasi

1. **Pembersihan Data Operasional UAT**: Seluruh tiket uji lama beserta dokumen, riwayat, dan pesan dibersihkan secara transaksional di PostgreSQL, serta berkas fisik dihapus dari storage bucket.
2. **Pencadangan Wajib Pra-Pembersihan**: Seluruh data operasional UAT, dokumen PDF, dan jejak audit dicadangkan secara terisolasi ke direktori arsip khusus sebelum pembersihan dijalankan.
3. **Data Master yang Dipertahankan**: Akun autentikasi staf, profil, 13 OPD, 2 layanan SPBE, dan 10 persyaratan checklist tetap utuh.
4. **Verifikasi READ-ONLY**: Seluruh hitungan pasca-pembersihan diverifikasi: tiket aktif = 0, jejak audit operasional = 0, storage berkas uji = 0, seluruh master data tetap utuh.

Revisi 7 — Integritas Dokumen Pengajuan Awal, Sanitasi Berkas & Persistensi Revisi

1. **Penegasan Integritas Dokumen Pengajuan Awal**: Menolak mutlak dokumen berformat metadata-only; mewajibkan buffer biner > 0 dan lolos magic bytes sebelum database disentuh.

2. **Sanitasi Nama Berkas**: Membatasi panjang maksimal 255 karakter dan membersihkan karakter kontrol/null byte tanpa merusak karakter Unicode alami.
3. **Persistensi Dokumen Revisi Publik**: Metadata berkas revisi disimpan di `application_documents` dengan `requirement_id: null` tanpa memalsukan checklist awal, serta didukung kompensasi pembersihan berkas jika terjadi kegagalan parsial.

Revisi 8 — Penguatan Konsistensi Status, History, Messages, dan Dokumen saat Submit Revision Gagal Sebagian

1. **Pencegahan Desinkronisasi Data Status**: Menerapkan rollback status kondisional dan alokasi ID eksplisit via `crypto.randomUUID()`.
2. **Pembersihan Selektif**: Blok catch hanya membersihkan dokumen dan riwayat milik sesi revisi tersebut.
3. **Proteksi Konkurensi**: Update maju ke `VERIFICATION` menerapkan filter konkurensi optimistik `.eq('current_status', 'REVISION_REQUIRED').select('id')` dan mendeteksi bentrok 0 baris (HTTP 409).
4. **Verifikasi Suite Pengujian**: Pengujian unit backend bertambah menjadi 226 tests (100% lulus).

Revisi 9 — Penguatan Keamanan Dokumen Unggahan: Validasi Base64 Ketat & Storage Object Key Berbasis UUID Acak

1. **Validasi Base64 Ketat**: Mengimplementasikan `parseAndValidateBase64` sesuai standar RFC 4648, menolak bit sisa non-kanonis dan trailing garbage.
2. **Storage Key Berbasis UUID Acak Server-Generated**: Format `${prefix}/${crypto.randomUUID()}.${safeExt}` tanpa menyertakan nama file asli di path Storage, mengeliminasi risiko traversal.
3. **Verifikasi Suite Pengujian Unit**: Total pengujian unit backend bertambah menjadi 247 tests (100% lulus, 15 file test).

Revisi 10 — Fitur Penerbitan Ulang Kode Pelacakan (Tracking Code Reissue) Atomik & Keamanan Kriptografis

1. **Peningkatan Entropi Keacakan (128-bit)**:
   - Generator kode pelacakan menggunakan `crypto.randomBytes(16).toString('hex').toUpperCase()`, menghasilkan 128 bit entropi kriptografis murni dengan format `TRK-<32-HEX-CHARS>`.
   - Diimplementasikan secara konsisten pada pembuatan tiket baru (`applications.service.ts`) dan penerbitan ulang (`admin-applications.service.ts`).
2. **Penyimpanan Hash SHA-256 & Audit Forensik**:
   - Basis data hanya menyimpan hash SHA-256 dari kode (`tracking_code_hash`).
   - Kode plaintext tidak pernah disimpan di database dan tidak pernah dicatat ke dalam `audit_logs`.
3. **Transaksi Atomik via PostgreSQL RPC (`reissue_tracking_code_atomic`)**:

- Dibuat migrasi `20261019161000_reissue_tracking_code_atomic.sql` yang mendefinisikan fungsi PostgreSQL `reissue_tracking_code_atomic`.
- Memperbarui hash kode dan mencatat audit log `TRACKING_CODE_REISSUED` dalam satu transaksi database ACID.
- Menghapus jalur fallback non-atomik di backend untuk menjamin ketiadaan mutasi tanpa audit.
- Fungsi menerapkan `SECURITY DEFINER` dengan `SET search_path = ''`, merujuk objek skema eksplisit, dan membatasi izin `EXECUTE` hanya kepada `service_role`.

4. **Otorisasi Khusus Superadmin & Modal Tampilan Satu Kali**:
   - Endpoint backend diproteksi secara otoritatif dengan `@Roles('SUPER_ADMIN')`.
   - Dibuat antarmuka modal dialog `ReissueTrackingCodeModal.tsx` di Web Admin yang mewajibkan pengisian alasan minimal 10 karakter, konfirmasi persetujuan, dan menampilkan kode baru hanya satu kali untuk disalin petugas.
   - Setelah penerbitan ulang berhasil, kode pelacakan lama otomatis tidak berlaku.
5. **Verifikasi Suite Pengujian Unit Komprehensif**:
   - Menambahkan pengujian mendalam pada `admin-applications.spec.ts` (35 tests) mencakup otorisasi role, validasi input, keberhasilan RPC, penanganan error RPC, dan integritas audit log.
   - Total pengujian unit backend bertambah menjadi **276 tests** pada **16 file test** (**100% lulus**).

---

3. Kebijakan Pelacakan Publik Tiket CLOSED (Analisis & Rekomendasi)

Saat ini, backend `tracking.service.ts` memblokir akses ke tiket `CLOSED` dengan melemparkan `403 Forbidden` (`TICKET_CLOSED`). Berikut adalah analisis kebijakan untuk pertimbangan masa depan:

Opsi Kebijakan	Deskripsi Perilaku	Dampak Keamanan & Privasi	Rekomendasi Arsitektur
Opsi A (Aktif Saat Ini)	Mengembalikan **HTTP 403 Forbidden** dengan kode `TICKET_CLOSED` dan pesan bahwa tiket telah selesai dan ditutup permanen.	**Sangat Aman**: Seluruh rincian dokumen dan interaksi tiket langsung terkunci total dari kanal publik.	Paling sesuai jika dokumen permohonan pasca-BAST bersifat rahasia antar-instansi.
Opsi B (Read-Only Summary)	Mengizinkan verifikasi (HTTP 200), namun antarmuka mengunci seluruh form kirim pesan/revisi dan hanya menampilkan ringkasan statis bahwa layanan telah selesai diserahterimakan.	**Aman Terkendali**: Pemohon OPD tetap dapat melihat bukti bahwa permohonan mereka telah ditutup secara sah tanpa membuka celah mutasi baru.	Direkomendasikan jika OPD membutuhkan konfirmasi visual berkelanjutan atas penyelesaian tiketnya.
Opsi C (Jendela Retensi 30 Hari)	Tiket `CLOSED` tetap dapat diakses secara read-only selama 30 hari kalender sejak tanggal penutupan, setelah itu akses otomatis menjadi 403 Forbidden.	**Seimbang**: Memberikan masa transisi bagi pemohon untuk mengarsipkan bukti layanan sebelum akses ditutup permanen.	Memerlukan penambahan kolom `closed_at` pada kueri validasi backend.

> **Status Saat Ini:** Tetap mempertahankan **Opsi A** sesuai instruksi pengguna, hingga ada arahan kebijakan formal tersendiri.

---

4. Temuan Audit yang Ditangguhkan (*Pending Backlog Items*)

Item berikut telah diidentifikasi dan ditandai sebagai penyempurnaan di masa depan tanpa mengubah stabilitas sistem saat ini:

1. **Sinkronisasi Redis untuk Rate Limiting**: Memindahkan penyimpanan counter kegagalan verifikasi tiket dari RAM ke Redis saat arsitektur beralih ke multi-server/load balancer.
2. **Pekerja Pemindaian Antivirus Berkas**: Menambahkan background worker pemindaian ClamAV untuk berkas unggahan KAK sebelum disahkan.
3. **Penyempurnaan Pesan Penolakan Tiket Closed pada Antarmuka**: Memberikan kartu banner khusus di UI publik yang secara ramah menginformasikan bahwa permohonan telah selesai diserahterimakan dan mengarahkan ke layanan pengaduan jika ada kendala pasca-rilis.